

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA0302	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Chalamala Sai Harshitha	Reg. No.:	192424346
Section / Batch:	BTech (AI&DS)	Date:	09/10/2026

Code of Conduct

I _C. Sai Harshitha (192424346)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _C. Sai Harshitha_

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name. Identify email addresses. Extract mobile numbers. Detect technical skills (Python, Java, SQL, Machine Learning, NLP). Extract years of experience. Generate a structured summary of the candidate's profile. Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Regular Expression

A Regular Expression is a pattern used to search, match and extract specific information from text.

In this program, Regex is used to identify structured information such as:

Information	Regex Pattern
Name	Name:\s*(.*)
Email	Email:\s*([\w.-]+@[\w.-]+\.\w+)
Mobile	Mobile:\s*(\d{10})
Experience	Experience:\s*(\d+)\s*years?

Algorithm

1. Start the program and accept the resume details as text input.
2. Use Regular Expressions to extract the candidate's name, email and mobile number.
3. Search the resume for the specified technical skills.
4. Extract the candidate's years of experience.
5. Display all extracted information as a structured profile.
6. Check whether the candidate has at least 2 years of experience and Python skill.
7. Display Eligible if both conditions are satisfied; otherwise display Not Eligible.
8. Stop the program.

Python Code

```
import re  
resume = input("Enter resume details:\n")
```

```

name = re.search(r"Name:\s*(.*)", resume, re.I)
email = re.search(r"Email:\s*([\w.-]+@[ \w.-]+\.\w+)", resume, re.I)
mobile = re.search(r"Mobile:\s*(\d{10})", resume, re.I)
experience = re.search(r"Experience:\s*(\d+)\s*years?", resume, re.I)
skills_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
skills = []
for skill in skills_list:
    if re.search(r"\b" + re.escape(skill) + r"\b", resume, re.I):
        skills.append(skill)
print("\n===== CANDIDATE PROFILE =====")
print("Name      :", name.group(1) if name else "Not Found")
print("Email      :", email.group(1) if email else "Not Found")
print("Mobile     :", mobile.group(1) if mobile else "Not Found")
print("Skills     :", ", ".join(skills) if skills else "Not Found")
print("Experience :", experience.group(1) + " years" if experience else "Not Found")
print("\n===== ELIGIBILITY =====")
if experience and int(experience.group(1)) >= 2 and "Python" in skills:
    print("Status    : ELIGIBLE")
    print("Reason    : Minimum 2 years of experience and Python skill satisfied.")
else:
    print("Status    : NOT ELIGIBLE")
    print("Reason    : Minimum eligibility criteria not satisfied.")

```

Output

Enter resume details:

Name: Ananya Sharma Email: ananya.sharma@gmail.com Mobile: 9876543210 Skills: Python, Java, SQL, Machine Learning, NLP Experience: 4 years

===== CANDIDATE PROFILE =====

Name : Ananya Sharma Email: ananya.sharma@gmail.com Mobile: 9876543210 Skills: Python, Java, SQL, Machine Learning, NLP Experience: 4 years
 Email : ananya.sharma@gmail.com
 Mobile : 9876543210
 Skills : Python, Java, SQL, Machine Learning, NLP
 Experience : 4 years

===== ELIGIBILITY =====

Status : ELIGIBLE
 Reason : Minimum 2 years of experience and Python skill satisfied.

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword. Perform prefix-based searches. Perform suffix-based searches. Search using partial keywords. Perform case-insensitive searches. Display all matching product names. Generate a report showing the total number of matching products for each search.

Concepts Used

Search Type	Regex Pattern
Exact	^keyword\$
Prefix	^keyword
Suffix	keyword\$
Partial	keyword
Case-insensitive	re.I

Algorithm

1. Start the program and create a list of product names.
2. Accept the search keyword and search type from the user.
3. Create the appropriate Regular Expression pattern.
4. Search the product list using the generated pattern.
5. Apply case-insensitive matching using re.I.
6. Store and display all matching product names.
7. Display the total number of matching products.
8. Stop the program.

Python Code

```
import re
products = [
    "Apple iPhone 15", "Samsung Galaxy S24", "Apple MacBook Air", "Dell Laptop",
    "HP Laptop", "Samsung Galaxy Buds", "Apple AirPods", "Sony Headphones", "OnePlus Nord",
    "Realme Phone"
]
```

```

keyword = input("Enter search keyword: ")
print("\n1. Exact Search 2. Prefix Search 3. Suffix Search 4. Partial Search")
choice = input("Enter search type: ")
if choice == "1":
    pattern = r"^" + re.escape(keyword) + r"$"
elif choice == "2":
    pattern = r"^" + re.escape(keyword)
elif choice == "3":
    pattern = re.escape(keyword) + r"$"
else:
    pattern = re.escape(keyword)
matches = []
for product in products:
    if re.search(pattern, product, re.I):
        matches.append(product)
print("\n===== SEARCH RESULTS =====")
if matches:
    for product in matches:
        print(product)
else:
    print("No matching products found.")
print("Total Matching Products:", len(matches))

```

Output

```

Enter search keyword: laptop

1. Exact Search
2. Prefix Search
3. Suffix Search
4. Partial Search
Enter search type: 4

===== SEARCH RESULTS =====
Dell Laptop
HP Laptop
Total Matching Products: 2

```

3. A university is developing an automated online registration portal to verify student information before course enrolment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number. Validate the institutional email address. Validate the course code. Validate the semester information. Validate the student's mobile number.

Display appropriate validation messages for each field. Generate a final registration status report indicating whether the registration is successful.

Regular Expressions Used

Field	Pattern
Register Number	<code>^22[A-Z]{2}\d{3}\$</code>
Institutional Email	<code>^[w.-]+@simats\.edu\.in\$</code>
Course Code	<code>^DSA\d{2}\$</code>
Semester	<code>^[1-8]\$</code>
Mobile Number	<code>^[6-9]\d{9}\$</code>

Algorithm

1. Start the program and accept the student's registration details.
2. Validate the register number using a Regular Expression.
3. Validate the institutional email address.
4. Validate the course code and semester.
5. Validate the student's mobile number.
6. Display an appropriate validation message for each field.
7. Check whether all fields are valid and display the final registration status.
8. Stop the program.

Python Code

```
import re
reg_no = input("Enter Register Number: ")
email = input("Enter Institutional Email: ")
course = input("Enter Course Code: ")
semester = input("Enter Semester: ")
mobile = input("Enter Mobile Number: ")
```

```
valid_reg = re.fullmatch(r"22[A-Z]{2}\d{3}", reg_no)
valid_email = re.fullmatch(r"[\w.-]+\@simats\.edu\.in", email)
valid_course = re.fullmatch(r"DSA\d{2}", course)
valid_semester = re.fullmatch(r"[1-8]", semester)
valid_mobile = re.fullmatch(r"[6-9]\d{9}", mobile)
print("\n===== VALIDATION RESULTS =====")
print("Register Number :", "Valid" if valid_reg else "Invalid")
print("Email          :", "Valid" if valid_email else "Invalid")
print("Course Code    :", "Valid" if valid_course else "Invalid")
print("Semester       :", "Valid" if valid_semester else "Invalid")
print("Mobile Number  :", "Valid" if valid_mobile else "Invalid")
if valid_reg and valid_email and valid_course and valid_semester and valid_mobile:
    print("\nRegistration Status: SUCCESSFUL")
else:
    print("\nRegistration Status: UNSUCCESSFUL")
```

Output

```
Enter Register Number: 22AI123
Enter Institutional Email: student@simats.edu.in
Enter Course Code: DSA03
Enter Semester: 5
Enter Mobile Number: 9876543210
```

```
===== VALIDATION RESULTS =====
Register Number : Valid
Email           : Valid
Course Code     : Valid
Semester        : Valid
Mobile Number   : Valid

Registration Status: SUCCESSFUL
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem Solving Approach	20	Logical, well structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____